Introduction to Knowledge Engineering – Assignment CO1

Q1 – Represent Domain Knowledge (Emergency Triage) [6 marks]

1.1 Knowledge items and their types

#	Knowledge item	Type	Why
K1	Blood pressure above 140/90 is called high.	Declarative	A plain fact about the domain.
K2	Check airway first, then breathing, then circulation.	Procedural	Describes how to do a task, step by step.
K3	A patient over 50 with chest pain and high blood pressure is Critical.	Heuristic	A rule of thumb that nurses learn from experience.
K4	A child under 5 with fever is Urgent.	Heuristic	Experience-based shortcut.

1.2 Predicate logic

- K1: $\forall x$ (Patient(x) $\wedge$ BP(x) > 140/90 $\rightarrow$ HighBP(x))
- K3: $\forall x$ (Patient(x) $\wedge$ Age(x) > 50 $\wedge$ ChestPain(x) $\wedge$ HighBP(x) $\rightarrow$ Priority(x, Critical))
- K4: $\forall x$ (Patient(x) $\wedge$ Age(x) < 5 $\wedge$ Fever(x) $\rightarrow$ Priority(x, Urgent))

Read K3 as: "For every patient x, if x is older than 50 and has chest pain and high BP, then x's priority is Critical."

1.3 Production rules and a frame

Production rules

- R1: IF bp > 140/90 THEN high_bp = yes
- R2: IF age > 50 AND chest_pain = yes AND high_bp = yes THEN priority = Critical
- R3: IF age < 5 AND fever = yes THEN priority = Urgent
- R4: IF no other rule gives a priority THEN priority = Routine

Frame for a patient

Slot	Value / default
Frame name	Patient

Slot	Value / default
name	(to be filled)
age	(to be filled)
chest_pain	default = No
bp	default = Normal
temperature	default = 37 °C
priority	if-needed procedure: apply R2, R3, R4
Instance: Ramesh	age = 58, chest_pain = Yes, bp = 150/95 → priority = **Critical**

1.4 Evaluating the three schemes

Criterion (what it means)	Predicate logic	Production rules	Frames
Representational adequacy (can it express the knowledge?)	High – very precise	Medium – best for rules of thumb	High for patient records and objects
Inferential adequacy (can it derive new knowledge?)	High – sound proofs	Medium – chaining only	Medium – inheritance and defaults
Inferential efficiency (is reasoning fast?)	Low – proofs are slow	High – simple matching	High – direct slot lookup
Acquisitional efficiency (is it easy to collect knowledge?)	Low – experts do not think in logic	High – experts think in IF–THEN	Medium – needs a good class design

Conclusion: rules suit the nurses' heuristics, frames suit the patient record, and logic suits a small, precise core of facts.

1.5 Two difficulties

1. **Vague / uncertain knowledge:** "high" BP or "severe" pain has no sharp limit, so borderline patients fall between rules.
2. **Incomplete knowledge:** an unconscious patient cannot report symptoms, so rules needing symptoms cannot fire, and the defaults (e.g. chest_pain = No) may be wrong.

Analysis Inquiry – Data, information, knowledge

- **Data** = a raw value: "150/95".
- **Information** = data with meaning: "Ramesh's blood pressure is high."
- **Knowledge** = a general rule that uses information to act: "A patient over 50 with chest pain and high BP needs immediate care."

Data and information describe one patient; knowledge applies to all patients and drives decisions.

Q2 – Acquire Knowledge from an Expert (Paddy Disease Advisory) [6 marks]

2.1 Roles and other sources

- **Domain expert:** the plant-pathologist (knows how to diagnose).
- **Knowledge engineer:** interviews the expert and converts the knowledge into rules.
- **End users:** farmers and agriculture extension officers.

Sources other than the expert: (1) crop-protection manuals and department bulletins, (2) records of past disease cases, (3) research papers, (4) extension officers' field experience.

2.2 Three elicitation techniques

Technique	Why it suits paddy disease diagnosis
Structured interview	Quickly collects clear facts: symptoms, weather limits, treatments.
Observation / think-aloud (protocol analysis)	The expert looks at real or photographed leaves and talks aloud, showing visual cues that are hard to explain in words.
Card sorting	The expert groups leaf photos into diseases, showing which differences he or she really uses.

2.3 Five follow-up interview questions

1. How many humid days make you suspect blast, and what temperature matters?
2. How do blast spots look different from brown spot or sheath blight spots?
3. What counts as "a lot" of nitrogen fertiliser (kg per acre)?
4. Which signs would make you rule blast out completely?
5. When you suspect nutrient deficiency, which nutrients do you check first, and what does the soil test confirm?

2.4 Production rules (CF = certainty factor, marks where certainty is involved)

Rule	IF	THEN	CF
R1	spots = brown oval with grey centre AND humid_days = several	disease = blast	0.6 ("I suspect" → uncertain)
R2	R1 fired AND spots on lower leaves AND nitrogen = high	blast becomes more certain	+0.5 ("more certain" → uncertain)

Rule	IF	THEN	CF
R3	whole leaf yellow AND spots = none	suspect = nutrient deficiency	0.7 ("I would think of" → uncertain)
R4	suspect = nutrient deficiency	action = ask for soil test	1.0 (fixed procedure)

Combining R1 and R2: 0.6 + 0.5 × (1 − 0.6) = **0.8**. The CF numbers are my assumptions and must be confirmed with the expert.

2.5 Bottleneck, difficulties and validation

- **Knowledge-acquisition bottleneck:** getting knowledge out of experts and into rules is slow, costly and difficult. It is the main barrier to building knowledge-based systems.
- **Two difficulties in this project:** (i) much of the diagnosis is visual "gut feeling" the expert cannot explain; (ii) different experts may disagree on thresholds (humidity days, nitrogen amount).
- **Validation:** (1) run the rules on past cases with known diagnoses, (2) have a second expert review the rules, (3) run a field trial, (4) measure how often the system agrees with the expert.

Analysis Inquiry – Tacit knowledge

Tacit knowledge is skill from experience that people cannot easily put into words (the expert "just sees" blast). Questions only reveal what the expert can explain. **Observation with think-aloud** helps most, because the expert is caught while diagnosing and the engineer notes the cues used.

Q3 – Reasoning with Production Rules (Loan Pre-screening) [8 marks]

Method used: rules are scanned in order R1 → R6; the first rule that can fire (and has not fired before) is fired; then the scan restarts from R1.

3.1 Case A – Forward chaining

Start facts: Salaried, Income_Stable, Credit_Score_Good, No_Defaults, Amount_Within_Limit.

Step	Rule fired	Fact added	Working memory afterwards
1	R1 (Salaried, Income_Stable)	Employment_Secure	start facts + Employment_Secure

Step	Rule fired	Fact added	Working memory afterwards
2	R2 (Credit_Score_Good, No_Defaults)	Good_Credit	… + Good_Credit
3	R3 (Employment_Secure, Good_Credit)	Low_Risk	… + Low_Risk
4	R4 (Low_Risk, Amount_Within_Limit)	Approve_Loan	… + Approve_Loan

R5 and R6 never fire. **Conclusion: the loan is approved.**

3.2 Case A – Backward chaining (goal: Approve_Loan)

- Goal **Approve_Loan** ← R4 needs Low_Risk and Amount_Within_Limit.
- Sub-goal **Low_Risk** ← R3 needs Employment_Secure and Good_Credit.
- Sub-goal **Employment_Secure** ← R1 needs Salaried ✓ and Income_Stable ✓ → proved. (R5 is not tried.)
- Sub-goal **Good_Credit** ← R2 needs Credit_Score_Good ✓ and No_Defaults ✓ → proved.
- Low_Risk proved. Amount_Within_Limit ✓ is a given fact.
- **Approve_Loan is proved.**

3.3 Case B – Forward chaining

Start facts: Self_Employed, Income_Stable, Has_Collateral, Credit_Score_Good, No_Defaults, Amount_Exceeds_Limit.

Step	Rule fired	Fact added
1	R2 (R1 fails because Salaried is missing)	Good_Credit
2	R5 (Self_Employed, Income_Stable, Has_Collateral)	Employment_Secure
3	R3	Low_Risk
4	R6 (Low_Risk, Amount_Exceeds_Limit)	Refer_To_Manager

R4 cannot fire because Amount_Within_Limit is not true. **Approve_Loan cannot be derived.** The system concludes: the applicant is low risk, but the amount is above the limit, so the case is **referred to the manager**.

3.4 Case B – Backward chaining (goal: Approve_Loan)

Approve_Loan ← R4 needs Low_Risk and Amount_Within_Limit. Low_Risk is proved through R3, R5 and R2. But Amount_Within_Limit is not a known fact and no rule concludes it, so the system must **ask the user: "Is the loan amount within the limit?"** The answer is No, so the goal **fails**. The system can then try the goal Refer_To_Manager through R6.

3.5 Two conflict-resolution strategies

1. **Rule order (priority):** when several rules can fire, pick the first one in the list. Simple and predictable, but results depend on how rules are ordered. **This is the one I used.**
2. **Specificity:** pick the rule with the most conditions, so special cases override general rules.

(Refraction – never fire the same rule twice on the same facts – was also used to avoid loops.)

3.6 Contradictory, redundant and circular rules

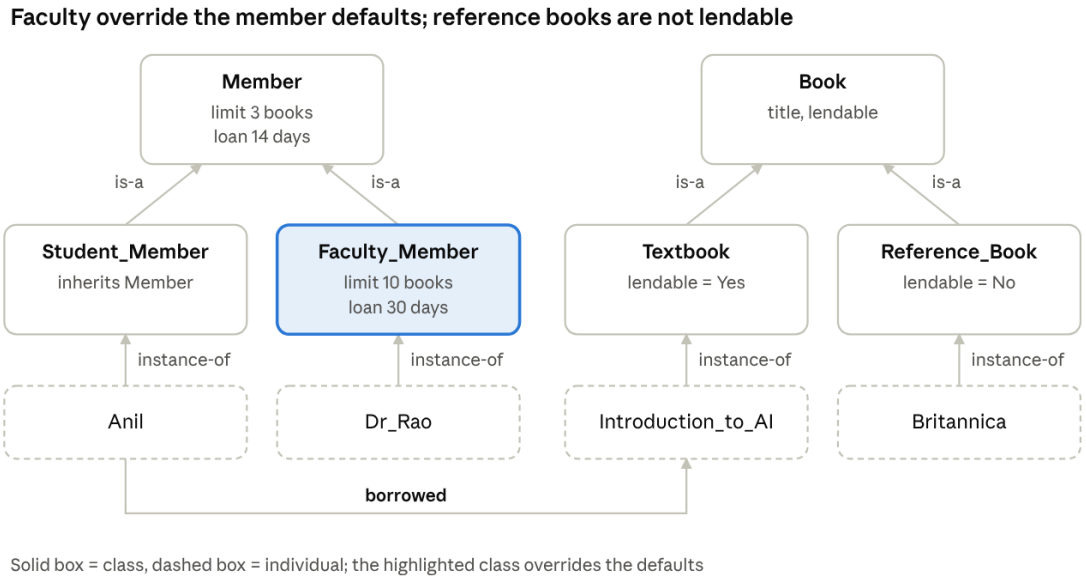
- **Contradictory rules** (one gives Approve, another Reject for the same facts): the answer depends on rule order and one of them is wrong. *Detect:* compare rules whose conditions overlap and check their conclusions.
- **Redundant rules** (duplicates, or one covered by another): waste time and make maintenance harder. *Detect:* look for identical or subsumed conditions.
- **Circular rules** (A → B and B → A): chaining can loop forever. *Detect:* draw a dependency graph and look for cycles, and test with sample cases.

Analysis Inquiry – Forward vs backward chaining

- **Forward chaining** (data-driven) is better when many facts arrive and you want every consequence: monitoring, configuration, or Case B's full derivation.
- **Backward chaining** (goal-driven) is better when you have one goal to check and want to ask only relevant questions: diagnosis, or "can this loan be approved?"

Q4 – Semantic Networks and Frames (Library Domain) [5 marks]

4.1 Semantic network



semantic network of the library domain

Links used: **is-a** (Student_Member → Member, Textbook → Book), **instance-of** (Anil → Student_Member), and **borrowed** (Anil → Introduction to AI).

4.2 Frames

Frame	is-a / instance-of	Slots
Member	–	name, membership_id, borrow_limit = 3 (default), loan_days = 14 (default)
Student_Member	is-a Member	inherits everything (no own values)
Faculty_Member	is-a Member	borrow_limit = 10, loan_days = 30 (own values)
Book	–	title, lendable
Textbook	is-a Book	lendable = Yes
Reference_Book	is-a Book	lendable = No
Anil	instance-of Student_Member	name = Anil, borrowed = Introduction to AI
Dr_Rao	instance-of Faculty_Member	name = Dr. Rao
Introduction to AI	instance-of Textbook	title
Britannica	instance-of Reference_Book	title

4.3 Answers using inheritance

To find a value, look at the object first, then go up through its parents and use the first value found.

- **How many books can Anil borrow?** Anil has no value → Student_Member has none → Member has 3. **Answer: 3 books** (he has already borrowed 1, so 2 more right now).
- **Dr. Rao's loan period?** Dr_Rao has none → Faculty_Member has loan_days = 30, found before Member's 14. **Answer: 30 days.**
- **Can Anil borrow 'Britannica'?** Britannica → Reference_Book has lendable = No. **Answer: No**, whatever Anil's limit is.

4.4 How a frame overrides a default

Member stores the general default (3 books, 14 days). Faculty_Member stores its own values (10 books, 30 days). Because a search starts at the lowest frame and stops at the first value found, the faculty values hide the defaults. Faculty are handled as an exception without changing Member or Student_Member.

4.5 Semantic network vs frames

	Semantic network	Frames
What it is	Nodes joined by labelled links	Objects with slots, defaults and procedures
Strength	Easy to read; shows relationships clearly	Handles defaults, exceptions and structure well
Limitation	Links have no standard meaning; weak for	Can become complex; inheritance

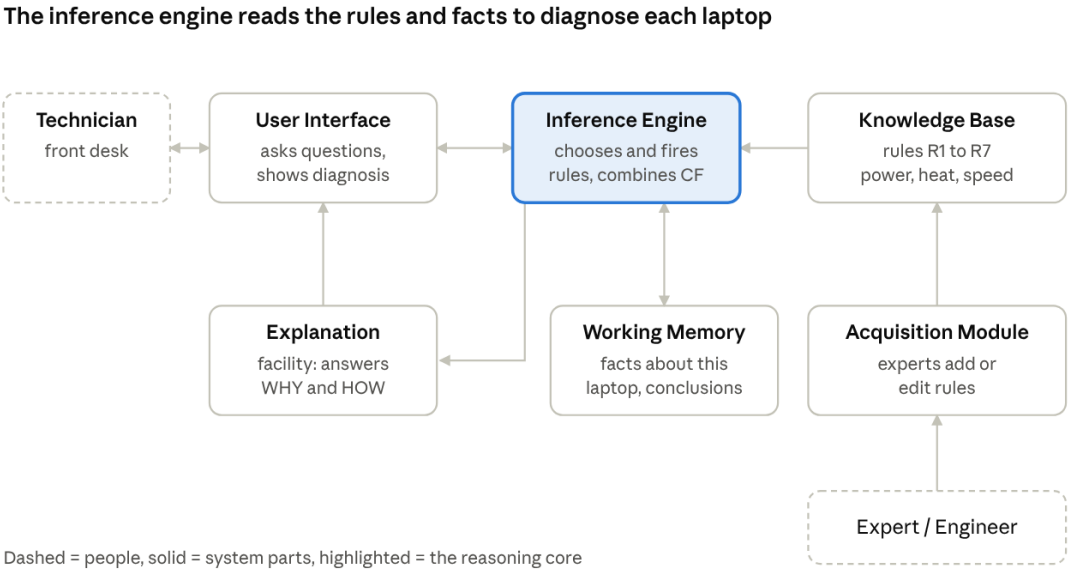
	Semantic network	Frames
	exceptions and relations with 3+ parts	conflicts; no formal meaning

Analysis Inquiry – A person who is both student and faculty member

The person inherits two different values (borrow_limit 3 and 10). The system does not know which to use – a **multiple-inheritance conflict**. It can be solved by (1) a class-precedence order (e.g. Faculty first), (2) a most-permissive or most-specific policy, or (3) giving that person's own frame a local value.

Q5 – Design an Expert System (Laptop Fault Diagnosis) [7 marks]

5.1 Architecture and role of each component



expert-system architecture · 6 components + 2 people

Component	Role in laptop diagnosis
Knowledge base	Stores the fault rules (power, overheating, slow performance).
Working memory	Holds the symptoms and conclusions for the current laptop.
Inference engine	Picks and fires rules and combines certainty factors.
User interface	Asks the technician questions and shows the diagnosis.
Explanation facility	Explains why a question is asked and how a result was reached.
Knowledge acquisition module	Lets the expert add or change rules.

5.2 Knowledge base (rules with CF)

Rule	IF	THEN	CF
R1	Not_Powering_On AND Charger_Light_Off	Faulty_Charger	0.8
R2	Not_Powering_On AND Charger_Light_On AND Battery_Swollen	Faulty_Battery	0.85
R3	Overheating AND Fan_Noisy	Dust_In_Fan	0.9
R4	Overheating AND Fan_Not_Spinning	Faulty_Fan	0.85
R5	Slow_Performance AND Fan_Noisy	Dust_In_Fan	0.6
R6	Slow_Performance AND High_Disk_Usage	Software_Or_Disk_Issue	0.7
R7	Dust_In_Fan	Recommend_Clean_Fan	1.0

5.3 Sample consultation

Facts supplied (with certainty): Overheating = 0.9, Fan_Noisy = 0.8, Slow_Performance = 0.7. Fan_Not_Spinning = No, High_Disk_Usage = No, Not_Powering_On = No.

Step	Rule fired	Result
1	R3	Dust_In_Fan, CF = 0.9 × min(0.9, 0.8) = 0.72
2	R5	Dust_In_Fan, CF = 0.6 × min(0.7, 0.8) = 0.42
3	Combine R3 and R5	CF = 0.72 + 0.42 × (1 − 0.72) = **0.8376 ≈ 0.84**
4	R7	Recommend_Clean_Fan, CF = 1.0 × 0.84 = 0.84

R1, R2, R4 and R6 do not fire because their conditions are No. **Recommendation: clean the fan and heat-sink (certainty about 0.84).**

5.4 Explanations

- **WHY** (system asks "Is the fan noisy?"): "I am asking because rule R3 can conclude Dust_In_Fan if the laptop is overheating and the fan is noisy. Overheating is already known."
- **HOW** (for Recommend_Clean_Fan): "R7 concluded it from Dust_In_Fan (CF 0.84). Dust_In_Fan was supported by R3 (0.72, from Overheating and Fan_Noisy) and R5 (0.42, from Slow_Performance and Fan_Noisy), combined into 0.84."

5.5 Combining certainty factors

1. For each rule: CF = rule CF × the smallest CF among its conditions (R3: 0.9 × 0.8 = 0.72; R5: 0.6 × 0.7 = 0.42).
2. When two rules support the same conclusion: CF = CF1 + CF2 × (1 − CF1) = 0.72 + 0.42 × 0.28 = 0.72 + 0.1176 = **0.8376**.

Two supporting rules raise confidence above either one alone, but never above 1. (This formula assumes the rules give fairly independent evidence.)

5.6 Expert system vs conventional program

	Expert system	Conventional program
Knowledge	Rules in a separate knowledge base	Mixed into the code
Problem type	Uncertain, rule-of-thumb reasoning	Step-by-step calculation
Explanation	Can explain WHY and HOW	Usually cannot
Changing it	Edit rules, no reprogramming	Change and recompile code

Limitation: an expert system is brittle – it fails on faults not covered by its rules, has no common sense, and its rules are costly to collect and update.

Analysis Inquiry – Why separate the knowledge base from the inference engine?

The expert can update rules without touching program logic; the same engine can be reused for other domains; rules stay readable and testable; and explanations become possible because the reasoning steps are visible rules.

Q6 – Ontologies and Knowledge Graphs (University Domain) [5 marks]

6.1 Ontology

- **Classes:** Person, Student, Professor, Course, Department.
- **Hierarchy:** Student is a subclass of Person; Professor is a subclass of Person.

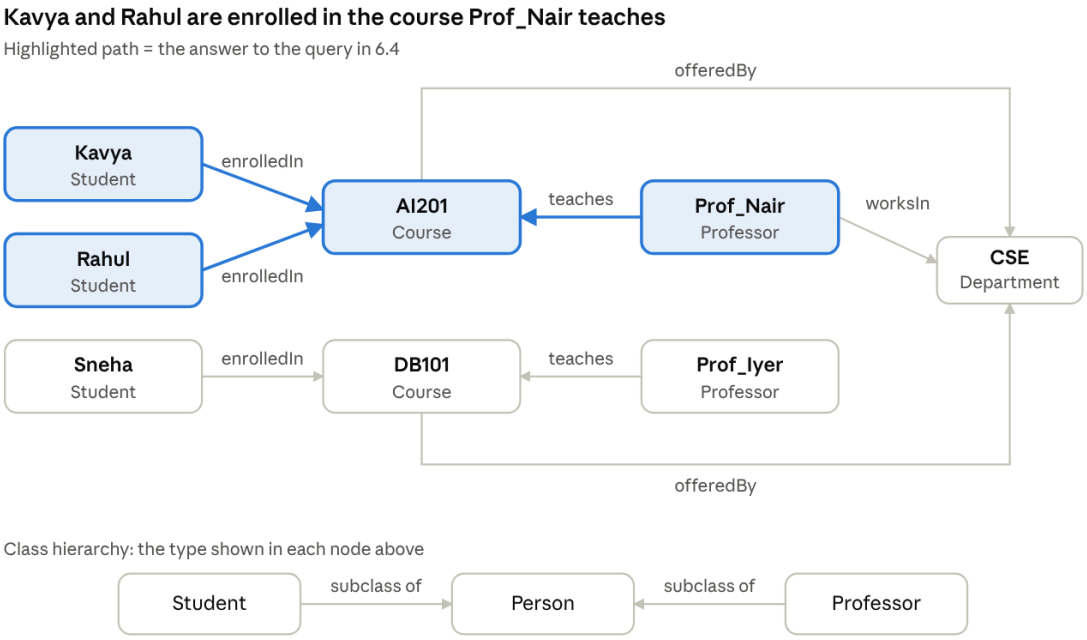
Property	Domain (from)	Range (to)
teaches	Professor	Course
enrolledIn	Student	Course
worksIn	Professor	Department
offeredBy	Course	Department

6.2 Triples (subject – predicate – object)

#	Subject	Predicate	Object
1	Prof_Nair	type	Professor
2	Prof_Nair	worksIn	CSE
3	Prof_Nair	teaches	AI201
4	AI201	offeredBy	CSE
5	Prof_Iyer	type	Professor
6	Prof_Iyer	teaches	DB101
7	DB101	offeredBy	CSE

#	Subject	Predicate	Object
8	Kavya	type	Student
9	Kavya	enrolledIn	AI201
10	Rahul	type	Student
11	Rahul	enrolledIn	AI201
12	Sneha	type	Student
13	Sneha	enrolledIn	DB101

6.3 Knowledge graph



university knowledge graph · 8 nodes, 13 triples

6.4 Graph query

Question: Which students are enrolled in courses taught by Prof_Nair?

Query: `{ Prof_Nair teaches ?c . ?s enrolledIn ?c }`

1. Pattern `Prof_Nair teaches ?c` matches triple 3, so **?c = AI201**.
2. Pattern `?s enrolledIn AI201` matches triples 9 and 11, so **?s = Kavya, Rahul**.
3. Sneha is enrolled in DB101, taught by Prof_Iyer, so she is not returned.

Answer: Kavya and Rahul.

6.5 Inference

- **From the hierarchy:** Kavya type Student, and Student is a subclass of Person, so **Kavya is a Person**, though this was never stored.
- **From the domain:** the domain of teaches is Professor, so anything that teaches a course must be a Professor.

6.6 Taxonomy vs ontology vs knowledge graph

	Taxonomy	Ontology	Knowledge graph
Content	Hierarchy of terms (is-a only)	Classes, properties, rules	Real facts (instances) linked together
Reasoning	Inheritance only	Logical inference	Queries, plus inference with an ontology
Example	Student is a Person	teaches has domain Professor	Prof_Nair teaches AI201

Analysis Inquiry – Sharing, reuse and a limitation

An ontology gives everyone the same agreed vocabulary, so different databases and applications mean the same thing by "Student" or "teaches". Data can then be combined, reused in new applications, and extended by inference. **Limitation:** building and maintaining an ontology is costly, and people often disagree on how to model concepts.

Q7 – Diagnose and Improve a Knowledge Management System (IT Help Desk) [13 marks]

Data used throughout (8 articles):

ID	Article words	Concept tag	Status
A1	reset password account login	Account_Access	current
A2	connect campus wifi network	Network	current
A3	install lab software license	Software	current
A4	vpn setup remote access old 2019	Remote_Access	retired
A5	vpn client remote access off-campus 2025	Remote_Access	current
A6	printer print queue lab	Printing	current
A7	exam portal login error	Exam_Portal, Account_Access	current
A8	student email setup mobile	Email	current

A. Diagnose the knowledge problem [2 marks]

Content (knowledge-quality) problems

- **Outdated / duplicate knowledge:** A4 (2019) and A5 (2025) both explain the VPN, and the old one is still searchable.
- **Vocabulary gap:** students write "internet", "sign in" and "eduroam"; articles say "wifi", "login" and "network".
- **Missing (tacit) knowledge:** support staff know "eduroam" is the campus Wi-Fi, but it was never written down.

System-design limitations (representation and retrieval)

- **Flat representation:** articles are just bags of words, with no concepts, synonyms or status.
- **Exact keyword matching:** if two articles tie, the earlier one wins, so an outdated article can be returned.

Evidence from example queries:

Query	Answer returned	Problem
no internet in hostel	nothing	word "internet" not in any article
cannot sign in	nothing	"sign in" ≠ "login"
vpn from home	A4 (retired)	A4 and A5 tie; old one comes first
eduroam problem	nothing	unknown term

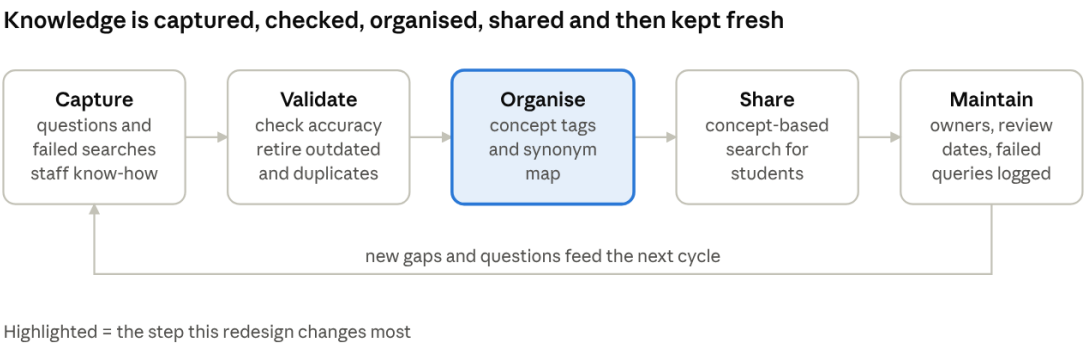
B. Define the knowledge objective [2 marks]

- **Knowledge to manage:** step-by-step solutions to common IT problems.
- **Users:** students (and help-desk staff who maintain the articles).
- **Representation:** articles with concept tags, a synonym list, and a status (current / retired).
- **Task:** return the one correct, current article for a free-text question.
- **Measurable objective:** on the 10 test queries, raise top-1 accuracy to **at least 80 %**, return **no retired article**, and leave **at most 10 %** of queries unanswered.

C. Select and redesign the method [3 marks]

- **Method chosen:** a light ontology – concept tags plus a synonym map – with article status, instead of plain keyword search.
- **Why:** most failures come from vocabulary mismatch and outdated content, which concept tagging and status control fix directly. With only 8 articles and one question type, a full knowledge graph or expert system would be too heavy, and this approach is cheap to maintain. The synonym map also writes down the staff's hidden knowledge.
- **Modification:** (1) retire A4; (2) map words to concepts ("internet", "wifi" → Network); (3) rank articles by number of matched concepts, then by specificity (matches ÷ number of tags).

Revised workflow:



revised knowledge-management workflow · 5 steps in a loop

D. Predict the outcome [1 mark]

Written before testing: retiring A4 and adding the synonym map should fix the queries that failed because of vocabulary mismatch or the outdated article, so top-1 accuracy should rise from about 60 % to **at least 80 %**, and no retired article should appear. Queries with words not in the synonym map (like "eduroam") should still fail.

E. Implement and evaluate [3 marks]

```
ARTICLES = [  # id, searchable words, concept tags, status
 ("A1","reset password account login",["Account_Access"],"current"),
 ("A2","connect campus wifi network",["Network"],"current"),
 ("A3","install lab software license",["Software"],"current"),
 ("A4","vpn setup remote access old 2019",["Remote_Access"],"retired"),
 ("A5","vpn client remote access off-campus 2025",["Remote_Access"],"current"),
 ("A6","printer print queue lab",["Printing"],"current"),
 ("A7","exam portal login error",["Exam_Portal","Account_Access"],"current"),
 ("A8","student email setup mobile",["Email"],"current"),
]
QUERIES = [  # query, correct article
 ("reset my password","A1"),("no internet in hostel","A2"),("wifi not connecting","A2"),
 ("vpn from home","A5"),("exam portal error","A7"),("cannot sign in","A1"),
 ("printer not printing","A6"),("set up college email on phone","A8"),
 ("install matlab","A3"),("eduroam problem","A2"),
]
SYN = {}
for words, c in [("password reset login sign account","Account_Access"),
 ("wifi internet network","Network"),("install software matlab","Software"),
 ("vpn remote home","Remote_Access"),("print printer","Printing"),
 ("exam portal","Exam_Portal"),("email mail","Email")]:
    for w in words.split():
        SYN[w] = c

def keyword_search(q):                    # ORIGINAL method
    best, best_score = None, 0
    for aid, text, _, _ in ARTICLES:
        score = sum(1 for w in q.split() if w in text.split())
        if score > best_score:            # ties keep the earlier article
            best, best_score = aid, score
    return best
```

```
def concept_search(q):                      # REVISED method
    want = {SYN[w] for w in q.split() if w in SYN}
    best, best_key = None, (0, 0)
    for aid, _, tags, status in ARTICLES:
        if status != "current":             # retired articles never returned
            continue
        m = len(want & set(tags))
        key = (m, m / len(tags))            # more matches, then more specific
        if m > 0 and key > best_key:
            best, best_key = aid, key
    return best

status = {a[0]: a[3] for a in ARTICLES}
for name, f in [("Keyword", keyword_search), ("Concept", concept_search)]:
    res = [(q, e, f(q)) for q, e in QUERIES]
    ok = sum(g == e for _, e, g in res)
    none = sum(g is None for _, _, g in res)
    old = sum(g is not None and status[g] == "retired" for _, _, g in res)
    print(name, "correct", ok, "/10, unanswered", none, ", outdated", old)
```

Output (I ran this program): Keyword: correct 6/10, unanswered 3, outdated 1. Concept: correct 9/10, unanswered 1, outdated 0.

#	Query	Correct	Keyword (original)	Concept-tagged (revised)
1	reset my password	A1	A1 ✓	A1 ✓
2	no internet in hostel	A2	none ✗	A2 ✓
3	wifi not connecting	A2	A2 ✓	A2 ✓
4	vpn from home	A5	A4 ✗ (retired)	A5 ✓
5	exam portal error	A7	A7 ✓	A7 ✓
6	cannot sign in	A1	none ✗	A1 ✓
7	printer not printing	A6	A6 ✓	A6 ✓
8	set up college email on phone	A8	A8 ✓	A8 ✓
9	install matlab	A3	A3 ✓	A3 ✓
10	eduroam problem	A2	none ✗	none ✗

Method	Top-1 accuracy	Unanswered	Outdated returned	Remaining errors
Keyword (original)	6/10 = 60 %	3	1	Queries 2, 4, 6, 10

Method	Top-1 accuracy	Unanswered	Outdated returned	Remaining errors
Concept-tagged (revised)	9/10 = 90 %	1	0	Query 10 ("eduroam" is not in the synonym map)

F. Justify the final design [2 marks]

- **Objective achieved?** Yes: accuracy rose from 60 % to 90 % (target ≥ 80 %), no retired article was returned, and 10 % were unanswered (target ≤ 10 %). The prediction in part D was correct, including the expected failure on "eduroam".
- **Limitations:** only 8 articles and 10 queries; I built the synonym map while looking at these same queries, so 90 % is probably too optimistic and must be re-tested on new queries; the synonym map needs manual upkeep; the scoring ignores word order and negation.
- **When another approach is better:** with hundreds of articles or free-form questions, use a statistical or embedding-based search or a knowledge graph; for step-by-step fault finding, use a rule-based troubleshooting system that asks questions.
- **Keeping knowledge up to date:** give every article an owner and a review date, log unanswered queries, and add new terms (like "eduroam") to the synonym map. This turns the staff's tacit knowledge into written knowledge.